



CHAPTER 5

단어 표현, 텍스트 분류, 그리고 텍스트 유사도

단어와 문서를 어떻게 수치로 표현하고 비교하는가?

학습 목표

- 단어 표현 (Word Representation)
 - 텍스트 수치화가 왜 필요한지 설명할 수 있다
 - 원-핫, BoW, N-gram, TF-IDF의 원리와 한계를 구분할 수 있다
- 텍스트 분류 (Text Classification)
 - 분류 문제의 유형과 지도/비지도 접근법을 설명할 수 있다
 - 전처리 → 벡터화 → 모델링 → 평가 파이프라인을 수행할 수 있다
- 텍스트 유사도 (Text Similarity)
 - 자카드·코사인·유클리디언·맨해튼 유사도를 계산·비교할 수 있다
 - 유사도를 문서 검색·중복 질문 탐지에 적용할 수 있다

목차

- 5.1 단어 표현
 - 텍스트 수치화 / 원-핫 인코딩 / BoW / N-gram
 - TF-IDF / 분산 표현(Word2Vec) / 방법 비교
- 5.2 텍스트 분류
 - 분류의 정의와 예시 / 지도 학습과 비지도 학습
 - 데이터 분석·전처리 / 모델링 / 성능 평가 지표
- 5.3 텍스트 유사도
 - 유사도의 정의 / 자카드·코사인·유클리디언·맨해튼
 - 유사도 기반 응용 (Quora Question Pairs) / 발전 방향

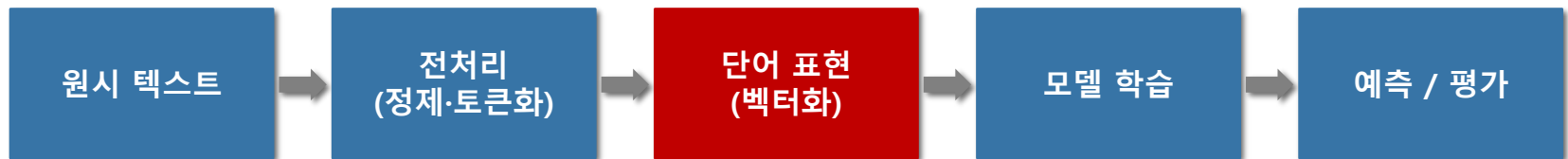
들어가며

- 컴퓨터는 문자를 이해하지 못한다 :
 - 사람은 "재미있는 영화였다"라는 문장을 읽고 곧바로 긍정적이라고 판단함
 - 그러나 컴퓨터는 문자열을 단순한 바이트 나열로만 인식할 뿐, 그 자체로는 어떠한 의미나 유사성도 계산할 수 없음
 - 머신러닝 모델(로지스틱 회귀, 랜덤 포레스트, 신경망 등)은 모두 숫자 벡터를 입력으로 받는 함수임
- 따라서 자연어 처리의 첫 단계는 항상 다음 질문에서 시작함 :

"단어와 문서를 어떻게 숫자로 바꿀 것인가?"

들어가며

- 자연어 처리의 기본 파이프라인 :
- 이번 주차는 아래 파이프라인 중 ②단어 표현과 ③모델링·평가 단계를 다룸



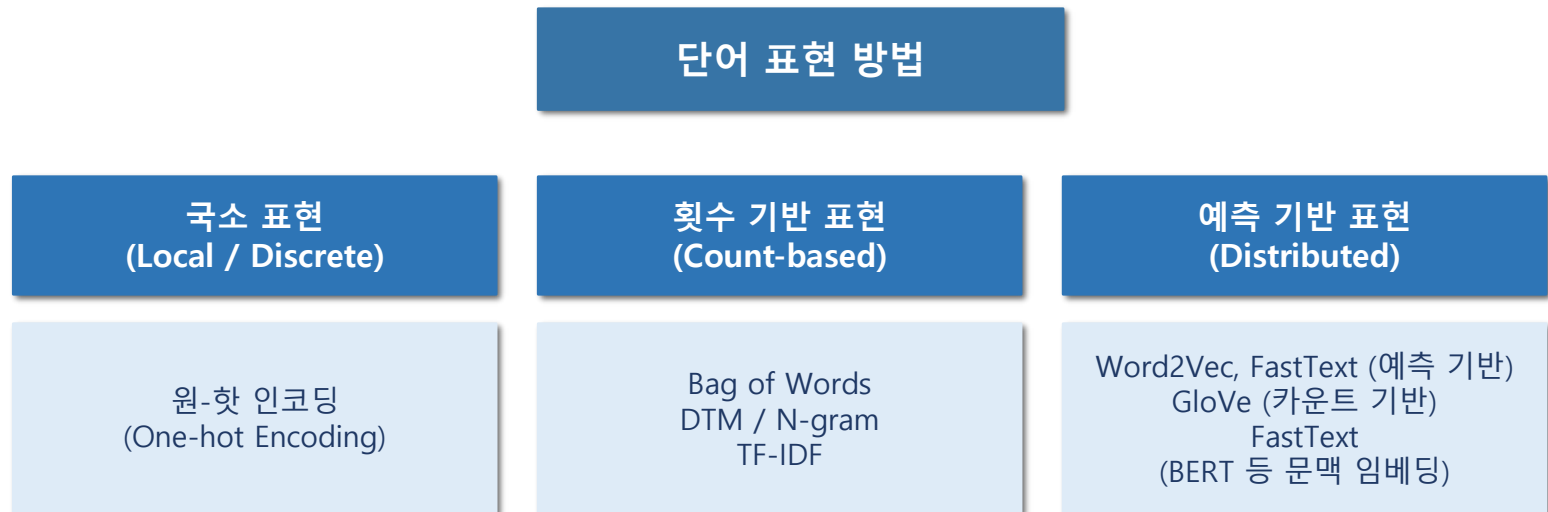
- 이번 주차의 핵심 : 텍스트 → 벡터 변환(단어 표현)이 이후 모델 성능의 상한을 크게 좌우함

5.1 단어 표현

- 단어 표현 (Word Representation) :
 - 텍스트를 구성하는 기본 단위인 단어를 컴퓨터가 연산할 수 있는 숫자 또는 벡터 형태로 바꾸는 모든 방법을 통칭함
 - 단어 임베딩(Word Embedding), 단어 벡터(Word Vector)라고도 부름
- 좋은 단어 표현이 갖추어야 할 조건 :
 - ① 의미가 비슷한 단어는 벡터 공간에서도 가까이 위치해야 함
 - ② 차원이 지나치게 크지 않아 계산과 저장이 효율적이어야 함
 - ③ 새로운 문서에 대해서도 동일한 규칙으로 변환할 수 있어야 함
 - ④ 문서 전체(문장·문단)를 표현하는 데까지 확장 가능해야 함

5.1 단어 표현

- 단어 표현 방법의 분류 :



- 5주차에서는 국소 표현과 횟수 기반 표현을 집중적으로 다루고, 예측 기반 표현은 개요만 소개함

5.1 단어 표현

- 원-핫 인코딩 (One-hot Encoding) :
 - 가장 기본적이고 직관적인 단어 표현 방법
 - 전체 말뭉치에 등장하는 서로 다른 단어를 모아 어휘 사전 (Vocabulary)을 만들고, 각 단어에 고유한 정수 인덱스를 부여함
 - 어휘 사전의 크기가 V 일 때, 각 단어를 길이 V 의 벡터로 표현하되 자기 자신의 인덱스 위치만 1이고 나머지는 모두 0으로 채움
 - 이렇게 하나의 값만 1(hot)이라는 뜻에서 원-핫(One-hot) 벡터라고 부름
 - 범주형 변수를 다루는 일반적인 머신러닝에서도 널리 사용되는 방식임

5.1 단어 표현

- 원-핫 인코딩 예시 :
 - 말뭉치 : "나는 자연어 처리를 배운다" → 어휘 사전 {나는, 자연어, 처리를, 배운다}

단어	인덱스	원-핫 벡터 (V = 4)
나는	0	[1, 0, 0, 0]
자연어	1	[0, 1, 0, 0]
처리를	2	[0, 0, 1, 0]
배운다	3	[0, 0, 0, 1]

- 어휘 사전이 커질수록 벡터의 길이도 그대로 함께 커진다는 점에 주목할 것

5.1 단어 표현

- 원-핫 인코딩의 한계 :
 - ① 차원의 저주 (Curse of Dimensionality) :
 - 실제 말뭉치의 어휘 사전은 수만~수십만 개에 이르므로 벡터 길이도 그만큼 길어짐
 - 거의 모든 값이 0인 희소 벡터(Sparse Vector)가 되어 메모리와 연산이 매우 비효율적임
 - ② 단어 간 유사도를 표현할 수 없음 :
 - "강아지"와 "개"는 의미가 매우 비슷하지만, 두 원-핫 벡터의 내적은 항상 0임
 - 즉 모든 단어 쌍이 서로 완전히 직교(orthogonal)하여 유사도가 언제나 0으로 계산됨
 - ③ 단어의 출현 빈도나 중요도 정보를 전혀 담지 못함

5.1 단어 표현

- Bag of Words (BoW) :
 - 단어의 순서는 무시하고 오직 출현 빈도(횟수)에만 집중하는 문서 표현 방법
 - 문서에 등장한 단어들을 가방(Bag)에 모두 넣고 흔든 뒤 개수만 세는 것에 비유됨
- BoW 생성 과정 :
 - ① 말뭉치 전체에서 각 단어에 고유한 정수 인덱스를 부여하여 어휘 사전을 만듦
 - ② 각 문서마다 어휘 사전 크기 V 의 벡터를 만들고, 해당 단어가 등장한 횟수를 기록함
 - ③ 결과적으로 문서 하나가 길이 V 인 하나의 빈도 벡터로 표현됨

5.1 단어 표현

- 문서 단어 행렬 (DTM, Document-Term Matrix) :
 - 여러 문서의 BoW 벡터를 행으로 쌓아 만든 행렬
 - D1 : "영화 재미 영화" / D2 : "영화 지루" / D3 : "재미 재미 추천"

문서	영화	재미	지루	추천
D1	2	1	0	0
D2	1	0	1	0
D3	0	2	0	1

- 행 = 문서, 열 = 단어 → 각 행이 곧 그 문서의 BoW 벡터가 됨
- 문서 간 비교, 유사도 계산, 분류 모델의 입력으로 곧바로 사용할 수 있음

5.1 단어 표현

- BoW 구현 : scikit-learn의 CountVectorizer

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = ["영화 재미 영화", "영화 지루", "재미 재미 추천"]

vect = CountVectorizer()          # 어휘 사전 생성기
X = vect.fit_transform(corpus)    # 학습 + 변환을 한 번에 수행

print(vect.get_feature_names_out()) # ['영화' '재미' '지루' '추천']
print(X.toarray())                # [[2 1 0 0] [1 0 1 0] [0 2 0 1]]
```

- fit_transform은 학습 데이터에만 사용하고, 검증·테스트 데이터에는 transform만 사용해야 함 (어휘 사전 누수 방지)

5.1 단어 표현

- Bag of Words의 한계 :
 - ① 단어의 순서 정보가 완전히 사라짐 :
 - "개가 사람을 물었다"와 "사람이 개를 물었다"가 동일한 벡터로 표현됨
 - 부정 표현("재미없다")이나 어순이 중요한 문장에서 치명적인 약점이 됨
 - ② 자주 등장하지만 의미 없는 단어가 큰 값을 차지함 :
 - "the", "is", "그리고", "하다" 같은 단어는 거의 모든 문서에 많이 등장함
 - 단순 빈도만 사용하면 이런 단어가 문서를 대표하는 것처럼 잘못 평가됨
 - ③ 여전히 희소 행렬이며, 어휘 사전 크기에 비례해 차원이 커짐

5.1 단어 표현

- N-gram :
 - BoW가 단어 순서를 잃어버리는 문제를 부분적으로 보완하는 방법
 - 연속된 N개의 토큰을 하나의 단위로 묶어서 어휘 사전을 구성함
 - $N = 1$ 이면 유니그램(Unigram), 2면 바이그램(Bigram), 3이면 트라이그램(Trigram)
- 예시 : "이 영화 정말 재미없다"
 - 유니그램 : {이, 영화, 정말, 재미없다}
 - 바이그램 : {이 영화, 영화 정말, 정말 재미없다}
 - 바이그램을 사용하면 "정말 재미없다"라는 부정 표현이 하나의 자질로 보존됨

5.1 단어 표현

- N-gram의 특징과 트레이드오프 :

구분	N이 작을 때 (1~2)	N이 클 때 (3 이상)
어휘 사전 크기	작음	기하급수적으로 커짐
문맥 반영	제한적	풍부함
희소성	상대적으로 낮음	매우 높음 (대부분 0)
일반화 성능	안정적	과적합 위험 증가

- 실무에서는 보통 `ngram_range=(1, 2)` 정도를 사용해 유니그램과 바이그램을 함께 씀
 - `CountVectorizer(ngram_range=(1, 2), min_df=2)`

5.1 단어 표현

- TF-IDF의 등장 배경 :
 - 단순 빈도(BoW)만 사용하면 모든 문서에 흔하게 등장하는 단어가 과대평가됨
 - 반대로 특정 문서에만 등장하는 단어일수록 그 문서를 잘 구별해 주는 단어임
- 핵심 아이디어 :
 - "어떤 문서에서 자주 등장하지만(TF ↑), 전체 문서 중에서는 드물게 등장하는(DF ↓) 단어일수록 그 문서를 대표하는 중요한 단어이다"
 - TF-IDF (Term Frequency - Inverse Document Frequency)는 이 직관을 가중치로 수식화한 것임

5.1 단어 표현

- TF-IDF의 구성 요소 :

- TF (Term Frequency) : 특정 문서 d에서 단어 t가 등장한 횟수
- DF (Document Frequency) : 단어 t가 등장한 문서의 개수
- IDF (Inverse Document Frequency) : DF의 역수에 로그를 취한 값

$$\text{IDF}(t) = \log \left(\frac{1 + N}{1 + \text{DF}(t)} \right) + 1 \quad (\text{scikit-learn 기본값, smooth_idf=True})$$

- N : 전체 문서 수, DF(t) : 단어 t를 포함한 문서 수
- 분모의 +1은 DF가 0일 때의 나눗셈 오류를, 분자의 +1은 모든 단어를 포함한 가상의 문서 1개를 더한 효과(smooth_idf)임
- 로그는 N이 커질 때 IDF가 지나치게 커지는 것을 억제함
- 모든 문서에 등장하면 $\text{DF} = N \rightarrow \log(1) + 1 = 1$ 이 되어 IDF가 최솟값 1을 가짐 (보정항 +1 덕분에 0으로 소멸하지 않음)

5.1 단어 표현

- TF-IDF 가중치 : **$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t)$**
 - 문서 d 안에서 단어 t가 갖는 상대적 중요도를 나타내는 실수 값
 - 값이 클수록 그 단어가 해당 문서를 잘 대표한다고 해석함
- 해석 :
 - $TF \uparrow, DF \downarrow \rightarrow TF\text{-}IDF \text{ 큼} \rightarrow$ 문서를 구별하는 핵심 단어 (예: "형태소", "퍼셉트론")
 - $TF \uparrow, DF \uparrow \rightarrow TF\text{-}IDF \text{ 작음} \rightarrow$ 흔한 단어 (예: "그리고", "the", "있다")
 - $TF = 0 \rightarrow TF\text{-}IDF = 0 \rightarrow$ 해당 문서에 등장하지 않는 단어

5.1 단어 표현

- TF-IDF 계산 예제 :
 - 문서 3개($N = 3$)로 구성된 말뭉치에서 각 단어의 TF와 DF를 먼저 구함
 - D1 : "영화 재미 영화" / D2 : "영화 지루" / D3 : "재미 재미 추천"

단어	TF(D1)	TF(D2)	TF(D3)	DF	IDF = $\log(4/(1+DF))+1$
영화	2	1	0	2	$\log(1.333)+1 = 1.288$
재미	1	0	2	2	$\log(1.333)+1 = 1.288$
지루	0	1	0	1	$\log(2.000)+1 = 1.693$
추천	0	0	1	1	$\log(2.000)+1 = 1.693$

- "영화", "재미"는 3개 중 2개 문서에 등장 → IDF가 최솟값 1에 가까워짐

5.1 단어 표현

- TF-IDF 계산 예제 (계속) :
 - TF와 IDF를 곱한 뒤 각 행을 L2 정규화하여 최종 TF-IDF 행렬을 완성함

문서	영화	재미	지루	추천
D1	0.894	0.447	0.000	0.000
D2	0.605	0.000	0.796	0.000
D3	0.000	0.836	0.000	0.549

- D2의 "지루", D3의 "추천"은 해당 문서에만 등장해 IDF가 가장 크며(1.693) 그 문서를 구별하는 단어가 됨
- 문서 수가 적어 IDF의 차이가 크지 않지만, 실제 말뭉치에서는 수천~수만 개의 문서를 사용하므로 값이 훨씬 세밀하게 분포함

5.1 단어 표현

- TF-IDF 구현 : scikit-learn의 TfidfVectorizer

```
from sklearn.feature_extraction.text import TfidfVectorizer

vect = TfidfVectorizer(
    min_df=2,           # 2개 미만 문서에 등장하는 단어는 제외
    max_features=5000,  # 상위 5000개 단어만 사용
    ngram_range=(1, 2), # 유니그램 + 바이그램
    sublinear_tf=True,  # TF에 1 + log(tf) 적용 (긴 문서 보정)
)
X_train = vect.fit_transform(train_texts)
X_test  = vect.transform(test_texts)    # transform만 사용
```

- scikit-learn은 기본적으로 각 문서 벡터를 L2 정규화하므로 코사인 유사도 계산이 곧 내적 계산이 됨

5.1 단어 표현

- TF-IDF의 장점 :
 - 구현이 간단하고 계산이 빠르며, 별도의 학습 데이터 없이 즉시 적용 가능함
 - sklearn 기본식은 IDF 하한이 1이므로 불용어 가중치가 0으로 소멸하지는 않음 → 상대적으로 낮아질 뿐이며 불용어 제거·max_df 설정은 여전히 필요함
 - 결과 해석이 쉬워 어떤 단어가 분류에 기여했는지 확인할 수 있음
 - 데이터가 적은 상황에서는 딥러닝 모델보다 더 좋은 성능을 내는 경우도 많음
- TF-IDF의 한계 :
 - 여전히 단어 순서와 문맥을 반영하지 못함
 - 동의어("영화"와 "무비")를 완전히 다른 자질로 취급함
 - 어휘 사전에 없는 새로운 단어(OOV)는 처리할 수 없음

5.1 단어 표현

- 분산 표현 (Distributed Representation) :
 - 희소 기반 표현의 희소성·유사도 문제를 해결하기 위해 제안된 방법
 - 단어를 수십~수백 차원의 조밀한 실수 벡터(Dense Vector)로 표현함
 - 분포 가설(Distributional Hypothesis)에 기반함 :
 - "비슷한 문맥에서 등장하는 단어는 비슷한 의미를 가진다"
 - 따라서 "강아지"와 "개"처럼 유사한 단어는 벡터 공간에서 가까이 배치됨
 - 벡터 연산으로 의미 관계를 표현할 수도 있음
 - 예 : $\text{vec}(\text{왕}) - \text{vec}(\text{남자}) + \text{vec}(\text{여자}) \approx \text{vec}(\text{여왕})$

5.1 단어 표현

- Word2Vec :
 - 대표적인 예측 기반 단어 임베딩 학습 방법으로 두 가지 구조가 있음

CBOw	Skip-gram
주변 단어들로부터 중심 단어를 예측 학습이 빠르고 자주 등장하는 단어에 유리	중심 단어로부터 주변 단어들을 예측 학습은 느리지만 희귀 단어 표현에 유리

- 이후 GloVe·FastText로 발전, 문맥을 반영하는 BERT로 이어짐 (이후 주차)

5.1 단어 표현

- 단어 표현 방법 비교 :

방법	차원	희소성	순서 반영	유사도 표현
원-핫 인코딩	V (매우 큼)	희소	×	불가 (항상 0)
Bag of Words	V	희소	×	부분적
N-gram	V^n (매우 큼)	매우 희소	Δ (국소적)	부분적
TF-IDF	V	희소	×	가능 (가중치)
Word2Vec	50~300	조밀	Δ	우수

- V = 어휘 사전의 크기 / 이번 실습에서는 BoW와 TF-IDF를 직접 구현하고 비교함

5.1 요약

- 핵심 정리 :
 - 컴퓨터는 문자를 이해하지 못하므로 텍스트를 반드시 수치 벡터로 바꾸어야 함
 - 원-핫 인코딩은 가장 단순하지만 차원이 크고 단어 간 유사도를 표현하지 못함
 - BoW는 단어 빈도로 문서를 표현하며 DTM 형태로 정리됨 (순서 정보는 손실)
 - N-gram은 연속된 N개 토큰을 묶어 국소적인 순서 정보를 일부 보존함
 - TF-IDF는 문서 내 빈도(TF)와 문서 간 희귀성(IDF)을 곱해 중요도를 계산함
 - 분산 표현(Word2Vec)은 조밀한 벡터로 의미적 유사도까지 표현함

5.2 텍스트 분류

- 텍스트 분류 (Text Classification) :
 - 자연어 처리 기술을 활용해 특정 텍스트가 사람들이 정한 몇 가지 범주 중 어느 범주에 속하는지 분류하는 문제
 - 분류해야 할 범주의 수에 따라 문제를 구분하기도 함
 - 이진 분류 (Binary Classification) : 2개의 범주로 구분하는 문제
 - 다중 범주 분류 (Multi-class Classification) : 3개 이상의 범주로 분류하는 문제
 - 다중 레이블 분류 (Multi-label) : 하나의 문서가 여러 범주에 동시에 속하는 문제
 - 앞 절에서 배운 단어 표현(BoW, TF-IDF)이 분류 모델의 입력 자질이 됨

5.2 텍스트 분류

- 텍스트 분류의 예시 :
 - 스팸 분류 (Spam Classification) :
 - 메일함에 들어갈 때마다 넘치는 스팸메일 때문에 짜증났던 기억이 있을 것임
 - 계속해서 들어오는 스팸메일 때문에 정작 필요한 메일은 확인하기 어려워짐
 - 자동으로 스팸 메일을 걸러서 사용자에게 일반 메일만 보여주는 것이 중요함
 - 이처럼 일반 메일과 스팸 메일을 분류하는 문제가 스팸 분류 문제임
 - 범주 : {정상(ham), 스팸(spam)} → 대표적인 이진 분류 문제

5.2 텍스트 분류

- 텍스트 분류의 예시 :
 - 감정 분류 (Sentiment Classification) :
 - 주어진 글이 긍정적인지 부정적인지 판단하는 문제로, 범주는 긍정 혹은 부정이 됨
 - 경우에 따라 중립 범주가 추가될 수도 있고, 긍정·부정의 정도에 따라 범주를 더 세분화할 수도 있음 (예 : 별점 1~5점)
 - 즉 문제에서 분류하려는 의도에 따라 범주가 정해지는 것임
 - 가장 대표적인 예는 영화 리뷰가 긍정적인지 부정적인지 판단하는 문제임
 - 실습에서는 한국어 영화 리뷰 감정 분류를 직접 수행함

5.2 텍스트 분류

- 텍스트 분류의 예시 :
 - 뉴스 기사 분류 (Topic Classification) :
 - 인터넷에는 매일 수많은 뉴스가 새로 생성됨
 - 뉴스 업체는 사용자가 원하는 뉴스를 골라 볼 수 있도록 범주를 잘 구분해 두어야 함
 - 이때 스포츠, 경제, 사회, 연예 등 다양한 주제의 기사를 주제에 맞게 분류해야 함
 - 자연어 처리 기술을 사용하면 뉴스 기사를 훨씬 효율적으로 분류할 수 있음
 - 범주가 3개 이상이므로 다중 범주 분류 문제에 해당함

5.2 텍스트 분류

- 텍스트 분류의 예시 :
 - 앞서 살펴본 예시 외에도 텍스트 분류에는 다양한 문제가 있음
 - 분류하는 단위를 글 전체로 하지 않고 각 단어를 한 단위로 분류하는 문제도 있음
 - 품사 분류 (POS Tagging) : 각 단어가 어떤 품사를 가지는지 분류하는 문제
 - 개체명 인식 (NER) : 각 단어가 인명·지명·기관명 중 무엇인지 분류하는 문제
- 그렇다면 텍스트 분류 문제는 어떻게 해결해야 할까?
 - 큰 기준으로 보면 지도 학습 또는 비지도 학습을 통해 해결할 수 있음

5.2 텍스트 분류

- 지도 학습을 통한 텍스트 분류 :
 - 지도 학습은 글(데이터)에 대해 각각이 속한 범주 값(라벨)이 이미 주어져 있음
 - 주어진 범주로 글을 모두 학습한 후, 학습 결과를 이용해 새로운 글의 범주를 예측함
 - 예를 들어 스팸 분류 문제를 지도 학습으로 해결하는 경우 :
 - ① 각 메일에 정상/스팸 라벨을 부여함 (레이블링)
 - ② 각 메일에서 자질(BoW, TF-IDF 등)을 추출함
 - ③ 모델이 예측한 결과와 라벨을 비교하면서 파라미터를 갱신함
 - ④ 학습된 모델로 새로운 메일이 스팸인지 아닌지 예측함

5.2 텍스트 분류

- 지도 학습 기반 문장 분류 알고리즘 :

알고리즘	특징
나이브 베이즈 분류	단어 독립 가정, 매우 빠르고 소량 데이터에 강함
로지스틱 회귀	선형 결합 + 시그모이드, 해석이 쉬움 (기준 모델)
서포트 벡터 머신	마진 최대화, 고차원 희소 데이터에 강함
랜덤 포레스트	여러 의사결정 트리의 결과를 평균 (배깅)
신경망 / 딥러닝	표현력이 크지만 많은 데이터와 연산량이 필요함

- 이번 주차 실습에서는 나이브 베이즈, 로지스틱 회귀를 직접 구현해 비교하고, scikit-learn이 설치된 환경에서는 랜덤 포레스트까지 추가로 비교함

5.2 텍스트 분류

- 비지도 학습을 통한 텍스트 분류 :
 - 지도 학습에서는 각 데이터가 어떤 범주인지에 대한 값(라벨)을 이미 가지고 있었음
 - 비지도 학습에서는 데이터만 존재하고 범주로 미리 나뉘어 있지 않음
 - 따라서 데이터의 특성을 찾아내서 적당한 범주를 만들어 각 데이터를 나눔
 - 특정한 정답 범주가 있는 것이 아니라 비슷한 데이터끼리 묶어주는 개념이므로 군집화(Clustering)라고도 불림
 - 대표적인 모델 : K-means Clustering, Hierarchical Clustering, DBSCAN
 - 문서 군집화, 토픽 모델링(LDA) 등에 활용됨

5.2 텍스트 분류

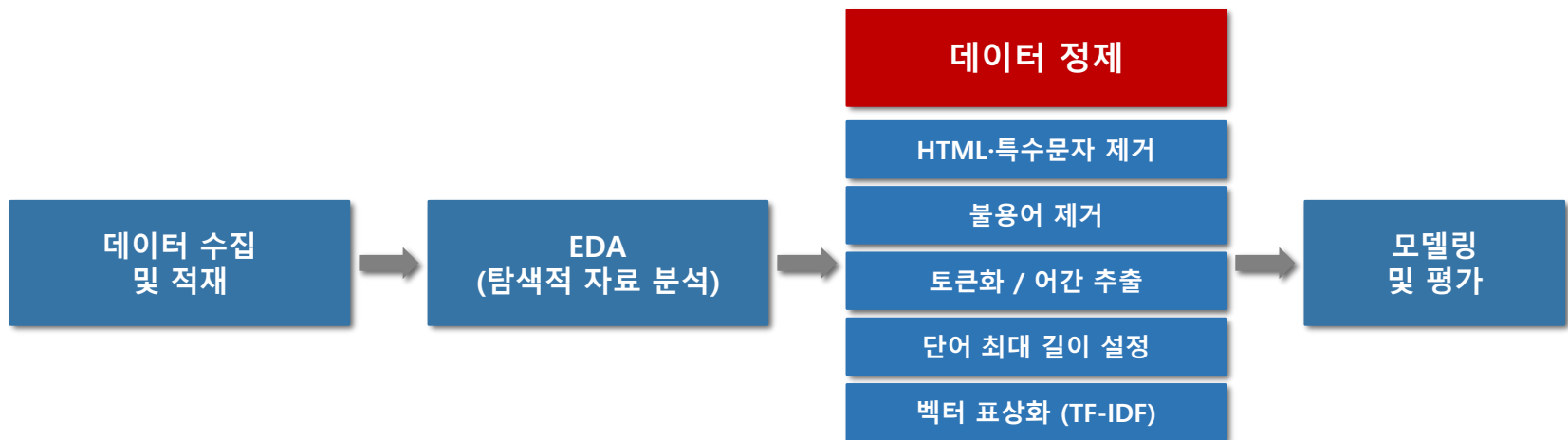
- 지도 학습과 비지도 학습의 선택 기준 :
 - 가장 큰 기준은 데이터에 정답 라벨이 있느냐 없느냐임

구분	지도 학습	비지도 학습
라벨	있음	없음
목표	정해진 범주로 예측	유사한 데이터끼리 군집화
대표 모델	로지스틱 회귀, SVM, RF	K-means, LDA
평가	정확도·F1 등 명확	실루엣 계수 등 간접 지표
비용	레이블링 비용이 큼	레이블링 불필요

- 일반적인 분류에는 지도 학습을, 정확한 범주 없이 묶기만 할 때는 비지도 학습을 사용

5.2 텍스트 분류

- 텍스트 분류 전체 처리 과정 :



5.2 텍스트 분류

- 데이터 분석 (EDA, 탐색적 자료 분석) :
 - 전처리 전에 데이터가 어떻게 구성되어 있는지 자세히 살펴보는 과정
 - 분석 결과를 바탕으로 어떤 전처리를 적용할지 결정함
- 주로 확인하는 7가지 항목 :
 - ① 데이터 파일 크기와 ② 데이터의 개수
 - ③ 각 문서의 문자 길이 분포와 ④ 단어 개수 분포
 - ⑤ 많이 사용된 단어 (워드클라우드, 빈도 상위 목록)
 - ⑥ 긍정·부정 등 라벨의 분포 (클래스 불균형 확인)
 - ⑦ 특수문자 및 대문자·소문자 비율

5.2 텍스트 분류

- 데이터 정제 (전처리) — 1단계 ~ 3단계 :
 - ① HTML 및 문장 부호 제거 :
 - 웹에서 수집한 리뷰에는 "
" 같은 HTML 태그와 특수문자가 섞여 있음
 - BeautifulSoup의 get_text()로 태그를 제거하고, re.sub()로 특수문자를 공백 처리함
 - ② 토큰화 (Tokenization) :
 - 문장을 의미 단위(단어, 형태소)로 나눔. 영어는 공백 기준, 한국어는 형태소 분석 사용
 - ③ 정규화 (Normalization) :
 - 대소문자 통일, 어간 추출(Stemming)·표제어 추출(Lemmatization) 수행

5.2 텍스트 분류

- 데이터 정제 (전처리) — 4단계 ~ 5단계 :
 - ④ 불용어 제거 (Stopword Removal) :
 - 불용어란 문장에서 자주 출현하지만 전체 의미에 큰 영향을 주지 않는 단어를 말함
 - 영어의 관사·전치사, 한국어의 조사·어미 등이 이에 해당함
 - 보통 라이브러리가 제공하는 불용어 사전(NLTK stopwords)을 사용함
 - NLTK 불용어 사전은 모두 소문자이므로 소문자 변환 후 제거해야 함
 - ⑤ 단어 최대 길이 설정 및 패딩 :
 - 모델 입력 길이를 통일하기 위해 긴 문서는 자르고 짧은 문서는 0으로 채움

5.2 텍스트 분류

- 데이터 정제 (전처리) — 6단계 : 벡터 표상화
- 정제된 텍스트를 5.1절에서 배운 방법으로 수치 벡터로 변환함

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split

X_tr_txt, X_te_txt, y_tr, y_te = train_test_split(
    texts, labels, test_size=0.2, stratify=labels, random_state=42)

vect = TfidfVectorizer(ngram_range=(1, 2), min_df=2)
X_tr = vect.fit_transform(X_tr_txt) # 학습 데이터로만 사전 구축
X_te = vect.transform(X_te_txt)    # 동일 사전으로 변환
```

- 학습 데이터로만 fit해야 검증 데이터의 정보가 새어 나가지 않음 (데이터 누수 방지)

5.2 텍스트 분류

- 모델링 ① 나이브 베이즈 분류 (Naive Bayes) :
 - 베이즈 정리를 이용해 문서가 각 범주에 속할 확률을 계산하는 확률 모델
 - 모든 단어가 서로 독립이라고 "순진하게(naive)" 가정하여 계산을 단순화함

$$P(c | d) \propto P(c) \times \prod P(w_i | c)$$

- $P(c)$: 범주의 사전 확률, $P(w_i | c)$: 범주 c 에서 단어 w_i 가 나올 확률
- 가장 높은 확률을 갖는 범주를 예측값으로 선택함
- 빠르고 소량 데이터에도 안정적이어서 기준(baseline) 모델로 자주 쓰임

5.2 텍스트 분류

- 모델링 ② 로지스틱 회귀 (Logistic Regression) :
 - 주로 이항 분류에 사용되며, 분류 문제에서 쓸 수 있는 가장 간단한 모델 중 하나임
 - 입력 자질의 선형 결합 결과에 로지스틱(시그모이드) 함수를 적용해 0~1 사이의 값을 얻고 이를 확률로 해석함
 - 값이 1에 가까우면 정답을 1로, 0에 가까우면 0으로 예측함
 - 학습된 가중치를 보면 어떤 단어가 긍정·부정에 기여했는지 직접 확인할 수 있어 해석력이 매우 높음
 - 고차원 희소 벡터(TF-IDF)와 궁합이 좋아 텍스트 분류의 표준 기준 모델로 사용됨

5.2 텍스트 분류

- 모델링 ③ 랜덤 포레스트 (Random Forest) :
 - 조금 더 복잡한 모델을 이용해 높은 성능을 얻고자 할 때 사용함
 - 먼저 의사결정 트리(Decision Tree)를 이해해야 함 :
 - 트리 구조 형태에서 각 노드는 하나의 질문이 되고, 답에 따라 다음 노드가 달라짐
 - 몇 개의 질문을 거친 뒤 최종 결과(범주)가 나오는 형태임
 - 랜덤 포레스트는 이러한 트리가 여러 개 모여 있는 형태임 :
 - 데이터와 자질을 무작위로 뽑아 서로 다른 트리를 여러 개 만듦
 - 각 트리의 결과를 종합(다수결·평균)해서 최종 결과를 도출함 → 배깅(Bagging)

5.2 텍스트 분류

- 성능 평가 ① 혼동 행렬 (Confusion Matrix) :
 - 예측 결과를 실제 정답과 교차하여 정리한 표

	예측 : 긍정	예측 : 부정
실제 : 긍정	TP (True Positive)	FN (False Negative)
실제 : 부정	FP (False Positive)	TN (True Negative)

- TP : 긍정을 긍정으로 맞춘 개수 / TN : 부정을 부정으로 맞춘 개수
- FP : 부정인데 긍정으로 잘못 예측 / FN : 긍정인데 부정으로 잘못 예측
- 클래스가 불균형할 때는 정확도만으로 판단하면 안 되므로 반드시 함께 확인해야 함

5.2 텍스트 분류

- 성능 평가 ② 주요 평가 지표 :

지표	수식	의미
정확도 (Accuracy)	$(TP+TN) / \text{전체}$	전체 중 맞게 예측한 비율
정밀도 (Precision)	$TP / (TP+FP)$	긍정이라 예측한 것 중 실제 긍정 비율
재현율 (Recall)	$TP / (TP+FN)$	실제 긍정 중 찾아낸 비율
F1 점수	$2PR / (P+R)$	정밀도와 재현율의 조화 평균

- 오탐(FP)이 치명적이면 정밀도를, 놓침(FN)이 치명적이면 재현율을 중시함

5.2 텍스트 분류

- 과적합(Overfitting)과 교차 검증 :
 - 과적합 : 학습 데이터에는 잘 맞지만 새로운 데이터에서는 성능이 떨어지는 현상
 - 텍스트 분류에서는 어휘 사전이 매우 크기 때문에 과적합이 특히 쉽게 발생함
- 대응 방법 :
 - min_df / max_features로 어휘 사전 크기를 제한함
 - L1·L2 규제(regularization)를 적용함 (LogisticRegression의 C 값 조정)
 - K-겹 교차 검증(K-Fold Cross Validation)으로 성능을 안정적으로 추정함
 - 학습 데이터를 K개로 나누어 K번 학습·검증하고 그 평균을 성능으로 사용함

5.2 텍스트 분류

- 사례 연구 : 영어 텍스트 분류 (워드 팝콘) :
 - "Bag of Words meets Bags of Popcorn"은 캐글의 대표적인 감정 분석 문제임
 - 인터넷 영화 데이터베이스(IMDB)의 영화 평점 데이터를 활용함
 - 각 데이터는 id, sentiment(긍정 1 / 부정 0), review 세 개의 열로 구성됨
 - 학습 데이터 25,000건, 테스트 데이터 25,000건 규모임
- 진행 순서 :
 - 불러오기 → EDA → 정제 → TF-IDF 벡터화 → 모델 학습 → 검증

5.2 텍스트 분류

- 한국어 텍스트 분류 시 유의점 :
 - 영어는 띄어쓰기 단위가 곧 단어에 가깝지만 한국어는 그렇지 않음
 - 한국어는 교착어이므로 어간에 조사·어미가 붙어 형태가 계속 달라짐
 - "영화가", "영화를", "영화는" → 모두 다른 토큰으로 처리되어 사전이 불필요하게 커짐
 - 따라서 형태소 분석기를 사용해 의미 단위로 분리하는 것이 일반적임
 - KoNLPy (Okt, Mecab, Komoran), Kiwi 등을 사용함
 - 실습에서는 환경 제약을 고려해 간단한 규칙 기반 토크나이즈를 사용함

5.2 요약

- 핵심 정리 :
 - 텍스트 분류는 문서를 정해진 범주 중 하나로 배정하는 문제임
 - 스팸 분류, 감정 분류, 뉴스 주제 분류, 품사 분류 등 응용 범위가 넓음
 - 라벨이 있으면 지도 학습, 없으면 비지도 학습(군집화)을 사용함
 - 전체 파이프라인은 수집 → EDA → 정제 → 벡터화 → 모델링 → 평가로 구성됨
 - 나이브 베이즈, 로지스틱 회귀는 빠른 기준 모델, 랜덤 포레스트는 성능 향상용
 - 정확도만 보지 말고 정밀도, 재현율, F1과 혼동 행렬을 함께 확인해야 함

5.3 텍스트 유사도

- 인공지능 스피커에 다음과 같이 질문한다고 생각해 보자 :

"이 노래 누가 만들었어?"

"지금 나오는 노래의 작곡가가 누구야?"

- 두 문장은 같은 의미이지만 인공지능 스피커는 단순히 다른 문장으로 인식함
- 따라서 각기 다른 대답을 준비해야 하지만, 비슷한 의미의 문장에 같은 대답을 연결할 수 있다면 훨씬 효율적임
- 이때 두 문장이 얼마나 유사한지 측정해야 하며, 이를 텍스트 유사도라고 함

5.3 텍스트 유사도

- 텍스트 유사도 (Text Similarity) :
 - 텍스트가 얼마나 유사한지를 수치로 표현하는 방식
 - 앞의 두 문장은 구조는 다르지만 의미가 비슷하므로 유사도가 높다고 판단할 수 있음
 - 유사도를 판단하는 척도는 매우 주관적이어서 데이터를 구성하기 쉽지 않고 정량화하는 데 한계가 있음
 - 이를 최대한 정량화해서 모델을 만드는 것이 중요함
- 정량화 접근 방법 :
 - 같은 단어의 개수를 사용하는 방법, 형태소 단위로 비교하는 방법, 자소 단위로 비교하는 방법 등 다양한 접근이 있음

5.3 텍스트 유사도

- 유사도 종류 :
 - 이번 절에서 다루는 네 가지 대표적인 유사도 측정 방법

자카드 유사도	코사인 유사도
집합의 교집합 / 합집합	두 벡터가 이루는 각도
유클리디언 유사도	맨해튼 유사도
두 점 사이의 직선 거리	좌표축을 따라 이동한 거리

- 자카드 유사도를 제외한 나머지는 먼저 문장을 벡터화(TF-IDF)한 뒤 계산함
- 자카드 유사도는 벡터화 없이 단어 집합만으로 바로 계산할 수 있음

5.3 텍스트 유사도

- 유사도 측정을 위한 사전 작업 : 벡터화
 - 유사도를 계산하려면 문장을 먼저 같은 차원의 수치 벡터로 만들어야 함
 - 여기서는 5.1절에서 배운 TF-IDF를 사용함

```
from sklearn.feature_extraction.text import TfidfVectorizer

sent = ("휴일인 오늘도 서쪽을 중심으로 폭염이 이어졌는데요, 내일은 반가운 비 소식이 있습니다.",
        "폭염을 피해서 휴일에 놀러왔다가 갑작스런 비로 인해 망연자실하고 있습니다.")

tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(sent)  # (2, V) 행렬
```

- 두 문장이 각각 하나의 벡터가 되고, 이후 모든 유사도는 이 벡터로 계산함

5.3 텍스트 유사도

- 자카드 유사도 (Jaccard Similarity) :
 - 두 문장을 각각 단어의 집합으로 만든 뒤, 두 집합의 교집합 크기를 합집합 크기로 나눈 값

$$J(A, B) = |A \cap B| / |A \cup B|$$

- 0과 1 사이의 값을 가지며, 1에 가까울수록 두 문장이 유사함
- 두 집합이 완전히 같으면 1, 공통 원소가 하나도 없으면 0이 됨
- 벡터화가 필요 없고 계산이 매우 빠르다는 장점이 있음
- 반면 단어의 등장 횟수(빈도)는 전혀 반영하지 못한다는 한계가 있음

5.3 텍스트 유사도

- 자카드 유사도 계산 예제 :
 - 유사도 측정 시 조사는 따로 구분하므로 두 문장 A, B는 다음과 같이 정의됨
 - $A = \{\text{휴일, 인, 오늘, 도, 서쪽, 을, 중심, 으로, 폭염, 이, 이어졌는 데요, 내일, 은, 반가운, 비, 소식, 있습니다}\}$
 - $B = \{\text{폭염, 을, 피해서, 휴일, 에, 놀러왔다가, 갑작스런, 비, 로, 인 해, 망연자실, 하고, 있습니다}\}$
 - 교집합 = {휴일, 을, 폭염, 비, 있습니다} → 5개
 - 합집합 → 25개 (17 + 13 - 5)
 - 따라서 자카드 유사도 = $5 / 25 = 0.20$

5.3 텍스트 유사도

- 코사인 유사도 (Cosine Similarity) :
 - 두 벡터 사이의 각도를 이용해 유사도를 측정하는 방법

$$\cos(\theta) = (A \cdot B) / (||A|| \times ||B||)$$

- 두 벡터의 내적을 각 벡터 크기의 곱으로 나눈 값
- 방향이 완전히 같으면 1, 직각이면 0, 정반대이면 -1의 값을 가짐
- TF-IDF 값은 항상 0 이상이므로 실제로는 0 ~ 1 사이의 값이 나옴
- 문서의 길이(벡터의 크기)에 영향을 받지 않는 것이 가장 큰 장점임
- 텍스트 유사도에서 가장 널리 사용되는 표준 척도임

5.3 텍스트 유사도

- 코사인 유사도가 선호되는 이유 :
 - 같은 내용을 담고 있어도 문서 길이가 다르면 벡터의 크기가 크게 달라짐
 - D1 : "영화 재미" $\rightarrow [1, 1]$ / D2 : "영화 영화 재미 재미" $\rightarrow [2, 2]$
 - 두 문서는 사실상 같은 내용이지만 유클리디언 거리로는 멀리 떨어진 것으로 계산됨
 - 코사인 유사도는 방향만 보므로 두 문서의 유사도를 1.0으로 정확히 판정함
- 주의할 점 :
 - 단어가 하나도 겹치지 않으면 의미가 아무리 비슷해도 유사도는 0이 됨
 - 이 한계는 단어 임베딩 기반 유사도로 보완할 수 있음

5.3 텍스트 유사도

- 유클리디언 거리 (유사도로 쓰려면 $\text{sim} = 1 / (1 + d)$ 로 변환) (Euclidean Similarity) :
 - 가장 기본적인 거리 측정 방식으로, 유클리디언 거리 또는 L2 거리라고 부름
 - N차원 공간에서 두 점 사이의 최단 직선 거리를 구하는 접근법

$$d(A, B) = \sqrt{\sum (a \cdot - b \cdot)^2}$$

- 거리이므로 값이 작을수록 두 문서가 유사함 (유사도와 방향이 반대)
- 값의 범위에 제한이 없어 1보다 큰 값이 얼마든지 나올 수 있음
- 제한이 없는 유사도 값은 비교·활용이 어려우므로 값을 제한해 주어야 함

5.3 텍스트 유사도

- 유클리디언 유사도의 정규화 :
 - 각 문장 벡터를 L1 정규화(모든 성분의 합이 1이 되도록 조정)한 뒤 거리를 계산하면 문서 길이의 영향을 줄일 수 있음 (성분이 0 이상이므로 유클리디언은 $0 \sim \sqrt{2}$, 맨해튼은 $0 \sim 2$ 범위)

```
import numpy as np
from sklearn.metrics.pairwise import euclidean_distances

def l1_normalize(v):
    return v / np.sum(np.abs(v), axis=1, keepdims=True) # 행(문장)별로 정규화

tfidf_dense = tfidf_matrix.toarray() # 희소 행렬 → 밀집 배열 (희소 행렬은 keepdims 미지원)
tfidf_norm_l1 = l1_normalize(tfidf_dense)
euclidean_distances(tfidf_norm_l1[0:1], tfidf_norm_l1[1:2])
```

- 정규화하면 문서 길이의 영향이 줄어들어 다른 유사도와 비교하기 쉬워짐

5.3 텍스트 유사도

- 맨해튼 유사도 (Manhattan Similarity) :
 - 맨해튼 거리(L1 거리)를 사용해 유사도를 측정하는 방법
 - 두 점 사이를 대각선으로 가로지르지 않고 좌표축을 따라 이동한 거리의 합
 - 바둑판 모양의 도시(맨해튼)에서 블록을 따라 이동하는 것에 비유해 붙은 이름임

$$d(A, B) = \sum |a \cdot - b \cdot|$$

- 유클리디언 거리와 마찬가지로 값에 제한이 없으므로 정규화가 필요함
- 차원이 매우 높은 희소 벡터에서는 유클리디언보다 안정적인 경우가 있음

5.3 텍스트 유사도

- 네 가지 유사도 비교 :

유사도	값의 범위	벡터화 필요	빈도 반영	길이 영향
자카드	0 ~ 1	불필요	×	작음
코사인	0 ~ 1 (TF-IDF)	필요	○	없음
유클리디언	0 이상 (무제한)	필요	○	큼
맨해튼	0 이상 (무제한)	필요	○	큼

- 자카드는 집합 크기 차이에 민감하므로 길이가 크게 다른 문서 비교에는 부적합하다. 일반적인 문서 비교에는 코사인 유사도를, 단순 단어 중복 확인에는 자카드를 사용함

5.3 텍스트 유사도

- 동일 문장에 대한 네 가지 유사도 계산 결과 (예시) :
 - 앞서 사용한 두 문장을 실습 노트북의 토큰나이저로 처리한 결과

유사도	계산 값	해석
자카드 ※ 값이 slide 56 과 다른 것은 토큰화 기준이 다르기 때문 - 토큰나이저가 바뀌면 유사도 값도 바뀐다	0.167	공통 토큰 4개 / 전체 24개
코사인	0.155	방향 기준으로는 낮은 유사도
유클리디언 (L1 정규화)	0.360	거리이므로 작을수록 유사
맨해튼 (L1 정규화)	1.641	거리이므로 작을수록 유사

- 척도마다 값의 스케일이 달라 절대값 비교는 의미가 없으며, 동일 척도 안에서의 순위 비교가 중요함

5.3 텍스트 유사도

- 응용 사례 : Quora Question Pairs
 - 목표 : 두 문장이 주어졌을 때 그 유사도를 측정하는 모델을 만드는 것
 - 캐글 대회 중 하나인 Quora Question Pairs 문제를 활용함
- 문제 소개 :
 - 쿼라는 질문과 답변을 주고받을 수 있는 사이트로 월 사용자가 약 1억 명에 달함
 - 매일 수많은 질문이 올라오는데 그중에는 분명 중복된 질문들이 포함되어 있음
 - 중복 질문을 잘 찾아낼 수 있다면 이미 작성된 좋은 답변을 바로 제공할 수 있음

5.3 텍스트 유사도

- 데이터 분석과 전처리 :
 - 데이터는 'id', 'qid1', 'qid2', 'question1', 'question2', 'is_duplicate' 열로 구성됨
 - 'id' : 각 행 데이터의 고유한 인덱스 값
 - 'qid1', 'qid2' : 각 질문의 고유한 인덱스 값
 - 'question1', 'question2' : 각 질문의 실제 내용
 - 'is_duplicate' : 0이면 중복이 아니고 1이면 중복인 질문 쌍
 - 데이터 분석을 통해 문장 길이 분포와 어휘 빈도를 파악하고 이를 전처리에 반영함

5.3 텍스트 유사도

- 데이터 분석과 전처리 (계속) :
 - ① 라벨 불균형 조정 :
 - 중복/비중복으로 나눈 뒤(loc 활용) 많은 쪽을 샘플링해 개수를 맞춤
 - ② 문자열 전처리 :
 - 정규 표현식(re.compile)으로 구두점.기호를 제거하고 소문자로 변환함
 - ③ 토큰나이징 및 패딩 :
 - 두 질문을 합쳐 공통 단어 사전을 만든 뒤 각 질문을 따로 인덱스로 변환함
 - 최대 길이를 정해 긴 문장은 자르고 짧은 문장은 패딩함

5.3 텍스트 유사도

- 모델링 : XGBoost 텍스트 유사도 분석 모델
 - 두 질문 문장 사이의 유사도를 측정해 중복 여부를 판단하는 모델을 만듦
 - XGBoost는 앙상블 기법 중 부스팅(Boosting)을 사용하는 방법임
- 앙상블 기법 :
 - 여러 개의 학습 알고리즘을 결합해 더 좋은 성능을 얻는 방법
 - 배깅(Bagging) : 여러 모델의 결과를 동등하게 취합함 (예 : 랜덤 포레스트)
 - 부스팅(Boosting) : 이전 모델이 잘못 예측한 부분에 가중치를 주어 순차적으로 학습을 이어감

5.3 텍스트 유사도

- XGBoost의 동작 원리 :
 - 트리 부스팅 기법을 활용한 모델임
 - 여러 개의 의사결정 트리를 사용하지만 단순히 결과를 평균 내지 않음
 - 결과를 보고 오답에 가중치를 부여한 뒤, 가중치가 적용된 오답을 맞히도록 다음 트리를 학습하는 과정을 반복함
 - 최종적으로 트리 부스팅 방식에 경사하강법을 적용해 최적화하는 방법임
 - 의사결정 트리 구성 시 병렬 처리를 사용해 연산량을 줄이고 빠르게 학습함
 - 입력 자질로는 두 질문의 벡터, 공통 단어 수, 자카드·코사인 유사도 등을 함께 사용함

5.3 텍스트 유사도

- 유사도의 실제 응용 :

응용 분야	활용 방식
검색 엔진	질의와 문서의 코사인 유사도가 높은 순으로 정렬
중복 문서 탐지	유사도가 임계값 이상인 문서 쌍을 중복으로 판정
추천 시스템	사용자가 본 문서와 유사한 문서를 추천
챗봇 / FAQ	입력 질문과 가장 유사한 FAQ의 답변을 반환
표절 검사	문서 간 유사도를 계산해 표절 의심 구간을 탐지

- 실습 3부에서 TF-IDF + 코사인 유사도 기반 미니 검색기를 직접 구현함

5.3 텍스트 유사도

- 횃수 기반 유사도의 한계와 발전 방향 :
 - 단어가 겹치지 않으면 의미가 같아도 유사도가 0이 됨
 - "자동차 가격"과 "차량 시세"는 의미가 같지만 TF-IDF 코사인 유사도는 0임
 - 어순과 문맥을 반영하지 못해 부정 표현이나 문법 구조를 구분하지 못함
- 발전 방향 :
 - 단어 임베딩(Word2Vec) 평균 벡터를 사용해 의미 유사도를 반영함
 - 문장 임베딩 모델(Sentence-BERT 등)로 문맥까지 반영한 유사도를 계산함
 - 실무에서는 TF-IDF(정확한 단어 일치)와 임베딩(의미 유사)을 함께 쓰는 하이브리드 검색이 널리 사용됨

5.3 요약

- 핵심 정리 :
 - 텍스트 유사도는 두 텍스트가 얼마나 비슷한지를 수치로 나타낸 값임
 - 자카드 유사도는 단어 집합의 교집합/합집합 비율로, 벡터화 없이 계산함
 - 코사인 유사도는 두 벡터의 각도를 사용하며 문서 길이의 영향을 받지 않음
 - 유클리디언·맨해튼 유사도는 거리 기반이므로 정규화가 필요함
 - 중복 질문 탐지, 검색, 추천, 표절 검사 등 다양한 분야에 응용됨
 - 단어가 겹치지 않으면 0이 되는 한계는 임베딩 기반 방법으로 보완함

전체 요약

- 5.1 단어 표현
 - 원-핫 인코딩 / Bag of Words / N-gram / TF-IDF / 분산 표현
- 5.2 텍스트 분류
 - 분류의 정의와 예시, 지도·비지도 학습, 전처리 파이프라인
 - 나이브 베이즈·로지스틱 회귀·랜덤 포레스트, 혼동 행렬과 평가 지표
- 5.3 텍스트 유사도
 - 자카드·코사인·유클리디언·맨해튼 유사도, 응용과 한계
- 다음 주차 예고
 - 텍스트 요약(TextRank·ROUGE), 텍스트 생성(마르코프 체인·n-gram), 그리고 벡터 표현

참고서적

- 전창욱, 최태균, 조중현, 신성진 지음, 「텐서플로 2와 머신러닝으로 시작하는 자연어 처리」, 위키북스
- 홉슨 레인, 하네스 막스 하프케, 콜 하워드, 「파이썬으로 배우는 자연어 처리 인 액션」, 제이펍
- 소햄 고시(Sohom Ghosh), 드와이트 거닝, 「예제로 배우는 자연어 처리 기초」, 에이콘
- 임희석, 「자연어처리 바이블 — 핵심이론·응용시스템·딥러닝」, 휴먼싸이언스
- scikit-learn User Guide : Working With Text Data

질문 및 정리

- 오늘 배운 내용을 스스로 점검해 봅시다 :
 - 원-핫 인코딩과 TF-IDF의 가장 큰 차이는 무엇인가?
 - BoW가 "개가 사람을 물었다"와 "사람이 개를 물었다"를 구분하지 못하는 이유는?
 - IDF에 로그를 취하는 이유는 무엇인가?
 - 클래스가 불균형할 때 정확도 대신 어떤 지표를 봐야 하는가?
 - 문서 길이가 서로 다를 때 코사인 유사도가 유리한 이유는?